Project 2: Smart Campus Navigation and Facility Analytics System

Smart Campus Navigation and Facility Analytics System

Important Instructions

- This project is designed for groups of three over six weeks: three weeks for Milestone 1 and three additional weeks for the Final submission.
- **[M1]** means required for Milestone 1 and must remain in the Final. **[Final]** means required only for the Final submission. The Final submission includes every M1 feature, and Final grading re-tests M1 features, so M1 defects should be fixed before the Final.
- Use the four instructor-provided data files (**data.zip)** **(https://unt.instructure.com/courses/147984/files/41454180?wrap=1)** **(https://unt.instructure.com/courses/147984/files/41454180/download?download_frd=1)** . Do not hard-code sample records, totals, paths, or report results. The instructor will also run your program on additional data files that use the same formats.
- Unless an exact output is specified, teams may choose clear output wording and formatting.
- Every team member must be able to explain any part of the submitted code.

1. Project Overview

Build a C++ program that loads campus data from text files and lets a user locate facilities, explore building connections, find routes, browse a department hierarchy, and view summary reports.

The project focuses on three data-structure topics:

- A custom hash table for facility lookup by ID.
- An undirected graph for campus buildings and walkways, including Breadth-First Search (BFS).
- A general tree for the academic department hierarchy.

This project requires one custom hash table, keyed by facility ID. Facility name and category searches may use a straightforward scan through the accepted facility records, so no additional hash tables are required.

2. Learning Objectives

1. Implement a custom string hash function, hash table, and one collision-handling strategy.
2. Represent an undirected graph and maintain adjacency relationships.
3. Implement BFS using a queue and a visited structure.
4. Implement a general tree in which a node can have any number of children.

5. Implement preorder, postorder, and level-order tree traversals.
6. Organize a modular C++ program, read and validate structured files, test edge cases, and collaborate through GitHub with visible contributions from every team member.

3. Team Organization and Contribution Evidence

The project divides naturally into three parts. A suggested split is shown below; shared work (file-loading utilities, menu, tests, documentation) belongs to everyone.

- **Member A:** facility records, custom hash table, ID search, name and category search, category report.
- **Member B:** campus graph, BFS reachability, shortest path, connectivity report.
- **Member C:** department tree, traversals, department statistics.

Phases

ID	Phase	Requirement
TEAM-1	**M1**	The GitHub repository has all three members as collaborators, and each member makes meaningful commits under their own account throughout the project.
TEAM-2	**M1**	Include CONTRIBUTIONS.txt listing each member's name, the modules each member owned, and shared work.
TEAM-3	**Final**	Update CONTRIBUTIONS.txt and add a short individual reflection from each member (3 to 5 sentences): what they built, what they reviewed, and one bug they found and fixed.
TEAM-4	**Final**	Document at least one code review in which a member reviews a teammate's module (for example, a pull request review or a short review note in the repository).
TEAM-5	**M1**	Individual scores may differ from the team score based on repository history, contribution statements, and participation in the demonstration.

4. Terminology and General Rules

Term	Meaning
Facility	Campus resource with an ID, name, building, and category.

Term	Meaning
Building / vertex	A building represented in the graph.
Connection / edge	A direct, undirected walkway between two buildings.
Degree	Number of distinct neighboring buildings.
Reachable	A building that can be reached from a start building; the start is reachable from itself.
Shortest path	A path with the fewest connections.
Isolated building	A building with degree zero.
Connected component	A maximal group of buildings in which every building can reach every other. An isolated building is a component by itself.
General tree	Hierarchy in which each node may have any number of children.
Root / leaf	Root is the top department; a leaf has no children. A root-only tree has one leaf.
Depth / height	Depth is edges from root (root depth 0); height is maximum depth (single-root tree height 0).
Load summary	One line printed after each file is loaded, giving the filename and the number of accepted and rejected records.

4.1 Matching and Ordering Rules

- **Case:** match names without regard to capitalization. Comparing ASCII letters is sufficient.
- **Whitespace:** trim whitespace around fields and user-entered names. Whitespace includes spaces, tabs, and the carriage return \r, so data files with Windows line endings must load correctly. Preserve spaces within names.
- **Internal spaces matter:** “Student Union” (two spaces) does not match “Student Union”.
- **Spelling:** preserve the spelling from the first accepted record for buildings, categories, and departments.

- **Building references:** a building named in connections.txt or facilities.txt is matched to buildings.txt ignoring case. Facilities and the graph display the building spelling from buildings.txt.
- **Facility IDs** are case-insensitive. After trimming, normalize every ID (from a file or typed by the user) to uppercase, and only then validate the format, hash, insert, compare, and display it. A valid ID is F followed by exactly three digits, such as F001. An ID written as f004 in a data file is accepted and stored as F004.
- **Comment lines:** a line is a comment only if its first non-whitespace character is #. A # elsewhere in a line is ordinary text (for example, “Room #5”).
- **Order:** buildings follow buildings.txt, neighbors follow accepted connection order, facilities follow facilities.txt, and tree siblings follow insertion order. Do not sort graph neighbors; this can change BFS order.
- **Rejection reasons:** if a record has more than one problem, report at least one valid reason. A suggested check order is field count, empty fields, ID format, unknown reference, then duplicate. Graders do not penalize the choice of reason.

5. Input Files

The instructor provides four files. Place them in data/ under the project root and load them in this order: buildings.txt, connections.txt, facilities.txt, departments.txt.

Files

File	Record format	Purpose
buildings.txt	BuildingName	Campus buildings.
connections.txt	BuildingA|BuildingB	Undirected building connections.
facilities.txt	FacilityID|FacilityName|BuildingName|Category	Facilities and locations.
departments.txt	DepartmentName|ParentName	Department hierarchy.

5.1 Rules for All Files

- Each record is one line; fields are separated by |. Ignore blank lines and comment lines.
- Read complete lines before splitting fields. Trim whitespace around each field; preserve internal spaces.

- Reject records with wrong field counts, empty required fields, duplicates, or invalid references, as applicable. Print a warning with filename, line number, and reason, then continue.
- If a file cannot be opened, print an error naming the file and continue without crashing; treat that file as empty.
- Connections and facilities may refer only to accepted buildings. A department parent must have been accepted on an earlier line.
- After each file, print a load summary line (Section 6.1, LOAD-4).
- Calculate all results from loaded data; do not hard-code sample totals or search results.

5.2 buildings.txt

Each non-comment line contains one nonempty building name. Names must be unique ignoring capitalization. Reject later duplicates.

```
# BuildingName
Library
Student Union
Science Building
Engineering Building
Business Building
Gym
Observatory
```

5.3 connections.txt

Each non-comment line contains BuildingA|BuildingB. Both names must identify accepted buildings. Reject self-connections and duplicates, including reversed duplicates (A|B and B|A). Connections are undirected: update both endpoints but count each connection once. Preserve neighbor insertion order.

```
# BuildingA|BuildingB
Library|Student Union
Library|Business Building
Student Union|Science Building
Student Union|Gym
```

Science Building|Engineering Building

Business Building|Gym

The sample contains seven buildings and six connections. Observatory is isolated.

5.4 facilities.txt

Each non-comment line contains FacilityID|FacilityName|BuildingName|Category.

- ID must be F plus exactly three digits (after uppercase normalization) and unique ignoring capitalization.
- Facility name and category must not be empty. Names may repeat.
- Building must identify an accepted building.
- Reject a later duplicate ID, retaining the first accepted record. Never overwrite a facility because another facility has the same name.
- A category spelling is recorded only from an accepted record, never from a record that is later rejected.

FacilityID|FacilityName|BuildingName|Category

F001|Main Reading Room|Library|Library

F002|Tutoring Center|Library|Tutoring Center

F003|Student Services Office|Student Union|Student Services Office

F004|Computer Lab A|Science Building|Computer Lab

F005|Room 101|Engineering Building|Classroom

F006|Computer Lab B|Engineering Building|Computer Lab

F007|Room 101|Business Building|Classroom

F008|Fitness Center|Gym|Recreation

F009|Room 205|Science Building|Classroom

F010|Research Help Desk|Library|Library

The sample contains two facilities named Room 101; a name search must return both.

5.5 departments.txt

Each non-comment line contains DepartmentName|ParentName. The first valid record is the root and must use NONE as parent. Every later department must name a parent accepted on an earlier line. Department names must be nonempty and unique ignoring capitalization. When adding a department from the menu, create the root without asking for a parent if the tree is empty; otherwise require an existing parent.

DepartmentName|ParentName

University|NONE

College of Engineering|University

College of Science|University

Computer Science|College of Engineering

Electrical Engineering|College of Engineering

Mathematics|College of Science

Physics|College of Science

5.6 Additional Data Files and Testing Files

- Keep the instructor-provided starter files in data/ unchanged in the final submission unless the instructor permits otherwise.
- Put your own test files in a separate testdata/ directory, and load them with the optional data-directory argument (NFR-10).
- The instructor will run your program on additional data files of the same format. They may include duplicate names that differ in capitalization, missing fields, malformed records, duplicate or reversed connections, unknown buildings, duplicate facility IDs, duplicate department names, unknown parents, Windows line endings, empty files, and missing files.

6. Required Functionality

6.1 File Loading

ID	Phase	Requirement
LOAD-1	**M1**	Load the four files in the required order from the data directory (default data/, relative to the working directory). Do not use absolute paths.
LOAD-2	**M1**	Apply the validation rules in Section 5. For every rejected record, print a warning with filename, line number, and reason, then continue.
LOAD-3	**M1**	If a file is missing or cannot be opened, print an error naming the file, treat it as empty, and continue.

ID	Phase	Requirement
LOAD-4	**M1**	After each file, print a load summary line with the filename and the number of accepted and rejected records. The facilities summary also reports hash table capacity, entries, and collisions (HASH-7).
LOAD-5	**M1**	Read complete lines with getline; handle blank lines, comment lines, extra whitespace, and Windows line endings (\r).

6.2 Facility Database

ID	Phase	Requirement
FAC-1	**M1**	Load valid facility records, retaining ID (uppercase), name, building (spelling from buildings.txt), and category.
FAC-2	**M1**	Display every accepted facility in file order, showing all fields and the total. If empty, display a clear message.
FAC-3	**M1**	Search by ID using the custom ID hash table. Trim and uppercase the entered ID first. Distinguish three outcomes: invalid format, valid but not found (show the normalized ID), and found.
FAC-4	**Final**	Search by complete facility name, ignoring case and outer whitespace. A simple scan is allowed. Display every exact match in file order and the match count. Partial-name matching is not required.
FAC-5	**Final**	Search by complete category, ignoring case and outer whitespace. A simple scan is allowed. Display matching facilities and the count.
FAC-6	**M1**	Reject duplicate facility IDs (ignoring case) and keep the first accepted record.

6.3 Custom ID Hash Table

ID	Phase	Requirement

ID	Phase	Requirement
HASH-1	**M1**	Implement your own hash-table class keyed by facility ID, supporting insertion, lookup, duplicate-key detection, and collision handling.
HASH-2	**M1**	Implement a custom string hash function that uses the characters of the normalized (uppercase) key and the table capacity to calculate a valid index. Do not use std::hash.
HASH-3	**M1**	Implement one collision strategy: separate chaining, linear probing, or quadratic probing. Explain your choice in the design document.
HASH-4	**M1**	Normalize the key to uppercase before hashing and storing. Reject duplicate keys case-insensitively and do not change the original entry.
HASH-5	**M1**	Do not impose an arbitrary fixed maximum number of facilities. Separate chaining is the recommended strategy because its chains can grow. The initial capacity is your choice, set in one named constant or constructor parameter. Teams using open addressing may reject an insertion when the table is full, but this limit is then part of the chosen design: document the capacity in the design document and print a clear warning whenever a record is rejected for this reason. Resizing is optional.
HASH-6	**M1**	ID lookup must use the hash table, not scan all facilities. Lookup must clearly indicate found or not found.
HASH-7	**M1**	Provide a diagnostic that reports table capacity, number of entries, and number of collisions encountered during loading (printed with the facilities load summary). Define in the design document what counts as a collision for your strategy (for example, an insertion into a nonempty bucket, or each occupied slot probed) and use that definition consistently. Count collisions only during accepted insertions; rejected duplicate IDs do not add to the count. Demonstrate at least one collision with your own test data directory in testdata/ whose IDs collide under your hash function and capacity, and record the diagnostic output in test_results.txt.

Standard utilities such as std::string, std::vector, and std::queue are allowed. Do not use std::unordered_map, std::map, std::unordered_set, or a library hash function or table in place of the required custom ID hash table.

6.4 Campus Graph

ID	Phase	Requirement

ID	Phase	Requirement
GRAPH-1	**M1**	Represent buildings as vertices and walkways as undirected, unweighted edges.
GRAPH-2	**M1**	Use an adjacency list or a dynamically sized adjacency matrix. An adjacency list is recommended. Document the choice.
GRAPH-3	**M1**	Display buildings in insertion order and neighbors in neighbor insertion order. For a building with no neighbors, display “(no connections)”. If there are no buildings, display a clear message.
GRAPH-4	**M1**	Display total buildings, total undirected connections counted once, and the degree of every building.
GRAPH-5	**M1**	Use the same graph representation for display, statistics, BFS, shortest path, and the Final connectivity report.
GRAPH-6	**M1**	Match building names typed by the user according to Section 4.1 (case-insensitive, trimmed, internal spaces preserved).

To simplify the project, adding buildings or connections through the menu is not required. These may be optional extensions, but are not part of the required grade.

6.5 BFS Reachability and Shortest Path

ID	Phase	Requirement
BFS-1	**M1**	Prompt for a starting building and use BFS (a queue and a visited structure) to display reachable buildings in visit order with distance in number of connections. Mark a building visited when it is added to the queue.
BFS-2	**M1**	Display reachable count out of total buildings. The start has distance 0 and counts as reachable.
BFS-3	**M1**	For an unknown start, display an error and return to the menu. Starting at an isolated building displays only that building.

ID	Phase	Requirement
BFS-4	**Final**	Prompt for start and destination; identify any unknown building. Use BFS to find a path with the fewest connections.
BFS-5	**Final**	Print the path one building per line and the number of connections. Same start and destination returns that building and 0 connections. If no route exists, report no path. For equally short routes, neighbor insertion order determines the selected path.

6.6 Department General Tree

ID	Phase	Requirement
TREE-1	**M1**	Implement a general tree; a node can have any number of children. A vector of child pointers is allowed.
TREE-2	**M1**	Load the tree from departments.txt using the Section 5.5 rules.
TREE-3	**M1**	Add a department from the menu. If the tree is empty, it becomes the root without a parent prompt; otherwise require an existing parent and reject duplicate names. Menu additions exist in memory only; departments.txt is never modified.
TREE-4	**M1**	Search by name ignoring capitalization. If found, display name, parent (None for the root), direct-child count, and depth.
TREE-5	**M1**	Display the hierarchy in preorder, one department per line, indented four spaces per depth. Show a clear message if empty.
TREE-6	**M1**	Implement preorder traversal and display the resulting sequence without indentation.
TREE-7	**Final**	Implement postorder and level-order traversals in addition to preorder. Level-order must use a queue. Visit siblings in insertion order.
TREE-8	**Final**	Display total departments, number of leaves, and tree height. For an empty tree, display a clear message with total 0, leaves 0, and height shown as N/A. A root-only tree has 1 department, 1 leaf, and height 0.

6.7 Reports

ID	Phase	Requirement
RPT-1	**Final**	Facility category report: for each distinct category, display facility count and percentage of all accepted facilities (one decimal place). Group category names ignoring capitalization and preserve the spelling of the first accepted category. Display the total count. If no facilities exist, state that none are available and show total zero; never divide by zero.
RPT-2	**Final**	Graph connectivity report: display total buildings, total connections, every building tied for highest degree, isolated buildings (or None), and the number of connected components. If there are no buildings, display a clear message with zero totals.

6.8 Menu and Interface

ID	Phase	Requirement
UI-1	**M1**	Show a repeating menu after loading files and printing warnings and summaries. Return to the menu after each operation until Exit.
UI-2	**M1**	The Milestone 1 menu provides the options in Section 6.9.
UI-3	**Final**	The Final menu provides all options in Section 6.9, including name and category search, shortest path, postorder and level-order, department statistics, and both reports.
UI-4	**M1**	Invalid menu input (blank, letters, or out-of-range number) must display an error and recover without crashing or looping forever. Recover from text at numeric prompts (clear the stream error and discard the rest of the line).
UI-5	**M1**	Every name prompt must accept spaces. A blank name displays an error and returns to the menu.
UI-6	**M1**	Every required operation must be accessible through the menu and demonstrable.
UI-7	**M1**	Use exactly the menu numbers listed in Section 6.9 (the M1 menu for Milestone 1 and the Final menu for the Final) so that scripted testing works. Option 0 exits cleanly.

6.9 Required Menus

Milestone 1 menu

1. View all facilities
2. Search facility by ID
3. View campus graph
4. Display graph counts and degrees
5. Find reachable buildings (BFS)
6. Add department
7. Search department
8. Display department hierarchy
9. Run preorder traversal
10. Exit

Final menu

1. View all facilities
2. Search facility by ID
3. Search facility by name
4. Search facility by category
5. View campus graph
6. Display graph counts and degrees
7. Find reachable buildings (BFS)
8. Find shortest path (BFS)
9. Add department
10. Search department
11. Display department hierarchy*
12. Preorder traversal**
13. Postorder traversal
14. Level-order traversal
15. Department statistics
16. Facility category report
17. Graph connectivity report
18. Exit

*Display the department tree with each department on a separate line, using four spaces of indentation per depth level.

**Display the departments in preorder traversal order as a sequence, without indentation.

7. Error Handling

Situation	Required behavior

Situation	Required behavior
Missing file	Print filename and continue without crashing.
Malformed line	Print filename, line number, and reason; skip it.
Duplicate record	Reject the later record and keep the earlier accepted record.
Unknown building or parent reference	Reject the record and identify the unknown reference.
Invalid ID format	Explain: F plus exactly three digits.
Valid ID not found	Print a not-found message with the normalized ID.
Unknown BFS or path building	Identify the unknown building and return to the menu.
No route between known buildings	Display a clear no-path message.
No facility search matches	Display a message and count zero.
Empty data structure	Display a clear empty-state message; do not crash.
Hash table full (open addressing)	Reject the insertion with a clear warning; do not crash or overwrite.
Blank name entered	Display an error and return to the menu.
Add department with unknown parent or duplicate name	Display an error naming the problem; leave the tree unchanged.

Situation	Required behavior
Invalid menu or numeric input	Display an error, recover, and return to the menu.

8. Implementation and Code Organization

8.1 Project Layout and Requirements

Project2/

|-- include/ Header files

|-- src/ Implementation files, including main.cpp

|-- data/ The four starter input files (unchanged)

|-- testdata/ Your own test files

|-- README.txt

|-- CONTRIBUTIONS.txt

|-- design_document.txt

|-- test_results.txt

|-- .gitignore

Requirements

ID	Phase	Requirement
NFR-1	**M1**	Use separate classes or clearly separated modules for facilities, custom hash table, campus graph, and department tree. Suggested names: Facility, HashTable, CampusGraph, DepartmentNode, DepartmentTree.
NFR-2	**M1**	Keep main.cpp focused on initialization, coordinating file loading, and the menu. Put data-structure operations in their classes or modules.
NFR-3	**M1**	Add a brief purpose and author comment to each source and header file, and brief comments explaining each function.

ID	Phase	Requirement
NFR-4	**M1**	Do not hard-code campus records or expected results.
NFR-5	**M1**	Avoid fixed-size arrays that impose arbitrary limits on facilities, buildings, connections, or departments.
NFR-6	**M1**	Standard containers such as std::vector, std::string, and std::queue are allowed. Implement the required ID hash table, BFS, and tree traversals yourself.
NFR-7	**M1**	Your project must compile and run successfully on the UNT CSE CELL machines before submission. You may develop your project on your personal computer or another development environment, but you are responsible for verifying that your final code compiles and runs correctly on CELL. **Important:** Please do not wait until the last minute or the final day to compile and test your project on the UNT CSE CELL machines. CELL machines may experience issues such as insufficient storage space or other technical problems. You are responsible for verifying that your project compiles and runs successfully on CELL before the submission deadline. Last-day emails reporting that you were unable to test or run your project on CELL due to a storage or similar issue will not be accepted as a reason for an extension or deadline exception. Please test your project on CELL well in advance of the deadline so you have sufficient time to address any issues. Document the exact build and run commands in README.txt (a Makefile is welcome but not required).
NFR-8	**M1**	The ZIP archive and the GitHub repository must contain matching versions of the required source, data, and documentation.
NFR-9	**M1**	.gitignore must exclude compiled executables, object files, and editor or IDE files. Do not submit executables.
NFR-10	**M1**	Accept an optional command-line argument naming the data directory (for example, ./campus testdata/case1). If it is omitted, use data/.
NFR-11	**Final**	Manage memory responsibly: release all dynamically allocated tree nodes and other heap memory in destructors (smart pointers are allowed). If a memory checker such as Valgrind is available, record a clean run in test_results.txt.

8.2 Documentation Files

- **txt:** purpose, exact build and run commands (including the optional data-directory argument), directory layout, and (Final) a module overview, the hash function, the collision strategy, and the graph representation.
- **txt:** about one page. Describe the hash function (including its formula), the collision strategy and why you chose it, how collisions are counted, the table capacity and full-table behavior, the graph representation and why you chose it, and the main classes and modules. In the Final, add the tree design, BFS and shortest-path approach, and the reports.
- **txt:** the tests from Section 10, recorded as test number, description, input, expected result, actual result, and pass or fail.
- **txt:** see TEAM-2 and TEAM-3.

9. Illustrative Expected Results

These are checks for the sample files. Calculate results in code; do not hard-code them.

Example 1: Load summaries (wording is flexible)

buildings.txt: 7 accepted, 0 rejected

connections.txt: 6 accepted, 0 rejected

facilities.txt: 10 accepted, 0 rejected (hash capacity C, entries 10, collisions N)

departments.txt: 7 accepted, 0 rejected

Example 2:

Facility ID search

Input	Expected outcome
F004	Found: Computer Lab A | Science Building | Computer Lab
f004	Same result as F004 (input normalized to uppercase)

Input	Expected outcome
004	Invalid format: an ID is F followed by exactly three digits
F999	Not found: no facility with ID F999

Example 3: Duplicate facility name (Final)

Searching “Room 101” returns:

F005 | Room 101 | Engineering Building | Classroom

F007 | Room 101 | Business Building | Classroom

Matches: 2

Example 4:

Graph counts and degrees

Building	Degree
Library	2
Student Union	3
Science Building	2
Engineering Building	1
Business Building	2
Gym	2
Observatory	0

Total buildings: 7. Total connections: 6. There are two connected components: one containing six buildings and one containing Observatory.

Example 5: BFS reachability

From Library:

Library (distance 0)

Student Union (distance 1)

Business Building (distance 1)

Science Building (distance 2)

Gym (distance 2)

Engineering Building (distance 3)

Reachable: 6 of 7 buildings

From Observatory:

Observatory (distance 0)

Reachable: 1 of 7 buildings

Example 6: Shortest path (Final)

Library to Engineering Building:

Library

Student Union

Science Building

Engineering Building

Number of connections: 3

Tie-breaking: Library to Gym has two shortest routes of 2 connections (through Student Union or through Business Building). Student Union is Library's first neighbor, and it reaches Gym first, so the expected path is:

Library

Student Union

Gym

Number of connections: 2

Library to Library prints Library and 0 connections. Library to Observatory reports that no path exists.

Example 7: Department hierarchy and statistics

University

College of Engineering

Computer Science

Electrical Engineering

College of Science

Mathematics

Physics

Total departments: 7

Leaf departments: 4

Tree height: 2

Empty tree statistics:

No departments are loaded.

Total departments: 0

Leaf departments: 0

Tree height: N/A

Example 8:

Facility category report (Final)

Category	Count	Percentage
Library	2	20.0%
Tutoring Center	1	10.0%

Category	Count	Percentage
Student Services Office	1	10.0%
Computer Lab	2	20.0%
Classroom	3	30.0%
Recreation	1	10.0%
Total	**10**	**100.0%**

Example 9: Graph connectivity report (Final)

Total buildings: 7

Total connections: 6

Highest degree (3): Student Union

Isolated buildings: Observatory

Connected components: 2

10. Required Testing

Test the sample files and additional cases, and record each test in test_results.txt (test number, description, input, expected result, actual result, pass or fail). Milestone 1 requires the tests tagged **[M1]**; the Final requires every test in this section.

10.1 File Loading

- **[M1]** Valid records, blank lines, comment lines, and records with too few or too many fields.
- **[M1]** A missing file.
- **[M1]** Duplicate building names that differ only by capitalization, and duplicate facility IDs that differ only by capitalization.
- **[M1]** Unknown building references (in connections and in facilities), duplicate and reversed connections, and unknown department parents.
- **[Final]** Empty files, empty required fields, self-connections, and duplicate department names that differ by capitalization.

- **[Final]** A file with Windows line endings, extra whitespace around fields, a lowercase ID in a data file, and a # inside a name.

10.2 Hash Table and Search

- **[M1]** Successful, lowercase, invalid-format, and not-found ID searches.
- **[M1]** At least one hash collision, using your own collision test data directory and the HASH-7 diagnostic, and a duplicate-key insertion attempt (confirming that it does not change the collision count).
- **[Final]** Multiple facilities with the same name; name and category searches that match, do not match, and ignore capitalization.

10.3 Graph and BFS

- **[M1]** BFS from a connected building and from an isolated building; BFS with an unknown start; a graph with multiple connected components.
- **[Final]** Shortest path where start equals destination, with an unknown start, with an unknown destination, and between buildings with no path.
- **[Final]** Two equally short routes, verifying that neighbor order determines the chosen path (Library to Gym in the sample data, plus a test file of your own).

10.4 Tree, Reports, and Menu

- **[M1]** Empty tree (adding the root without a parent prompt), root-only tree, searches for the root, a non-root, and a missing department, adding with an unknown parent, adding a duplicate, and preorder traversal.
- **[M1]** Invalid menu input, text at numeric prompts, names containing spaces, blank names, and repeated operations after errors.
- **[Final]** Postorder and level-order traversals; department statistics for an empty tree, a root-only tree, and the sample tree.
- **[Final]** Category report with capitalization differences and with no facilities; connectivity report with an isolated building and with a tie for highest degree.

11. Deliverables

11.1 Milestone 1

1. ZIP archive containing source and header files, all four input files, README.txt, CONTRIBUTIONS.txt, design_document.txt, test_results.txt, and .gitignore.
2. GitHub repository containing the same required files as the ZIP archive, with commits from all three members.
3. txt with purpose, build and run instructions, and directory layout.
4. Concise design_document.txt describing the custom hash function, collision strategy, graph representation, and main classes and modules.
5. txt with the [M1] tests from Section 10 and their results.

6. Working demonstration of all Milestone 1 features.

11.2 Final Submission

1. Updated ZIP archive containing the complete project.
2. Updated GitHub repository matching the ZIP archive.
3. Updated README with build and run instructions, module overview, hash function, collision strategy, and graph representation.
4. Updated design_document.txt, CONTRIBUTIONS.txt (with individual reflections), and test_results.txt (all Section 10 tests).
5. Complete implementation of all Milestone 1 and Final requirements, accessible through the Final menu.
6. Successful compilation and execution on CELL and a working demonstration.

12. Grading and Rubrics

12.1 Milestone 1 Rubric

Criterion (points)	Full credit	Partial credit	Little or no credit
File loading and validation 15 points	All four files load in the required order; every rejection rule for buildings, connections, facilities, and departments is enforced; each warning shows filename, line number, and reason; blank lines, comments, whitespace, and Windows line endings are handled; a missing file is reported without a crash; load summaries are correct; instructor test files load correctly.	Most files load and common rejections work, but some rules are missing (for example reversed duplicates, case-insensitive duplicates, or unknown parents), warnings lack line numbers or reasons, or load summaries are missing or wrong.	Frequent crashes or incorrect loading; validation largely absent; data or results appear hard-coded.
Facilities, custom hash table, and ID search 20 points	Custom string hash function and one correct collision strategy; IDs normalized to uppercase before hashing; duplicates rejected case-insensitively with the first record kept; lookup uses the table (no scan) and	The table works, but one or more of these is weak: collision not demonstrated or counting definition undocumented, duplicate handling incomplete, uppercase normalization	Uses std::unordered_map, std::map, std::hash, or a scan for ID lookup (no credit for hash-table items), or lookup is incorrect or crashes.

Criterion (points)	Full credit	Partial credit	Little or no credit
	distinguishes invalid, not found, and found; facilities display in file order with total; collision shown through the diagnostic using the team's own collision test data, with the counting definition documented; capacity and full-table behavior handled and documented.	inconsistent, capacity or full-table handling undocumented, or search outcomes not clearly distinguished.	
Campus graph 12 points	Dynamically sized adjacency list or matrix; edges updated at both endpoints and counted once; buildings and neighbors in insertion order; "(no connections)" shown; totals and degrees correct from loaded data; case-insensitive building lookup; same graph reused by later features.	Minor errors, such as a double-counted edge in one display, missing isolated-building message, or altered ordering in one place.	Directed edges, sorted neighbors, fixed-size arrays, or hard-coded output.
BFS reachability 10 points	BFS with a queue and a visited structure marked at enqueue; correct visit order and distances; reachable count out of total includes the start; unknown start reports an error; isolated start shows only itself; correct with multiple components.	Correct order but wrong distances or count, visited marked at dequeue, or edge cases partly handled.	Not BFS, incorrect order, or crashes on edge cases.
Department tree 15 points	General tree (any number of children); loads under Section 5.5 rules; add department (root without parent prompt when empty, existing parent required, duplicates rejected, memory only); search shows name, parent (None for root), child count, and depth; preorder hierarchy with four-space	The tree works, but indentation is wrong, search details are missing, empty cases are mishandled, or a parent is requested for the first department.	Binary-tree structure, hard-coded hierarchy, or incorrect traversal.

Criterion (points)	Full credit	Partial credit	Little or no credit
	indentation; clear empty-tree messages.		
Menu and input handling 8 points	Repeating menu with the required options and numbering; blank, text, and out-of-range input all recover; names with spaces work; blank names rejected; returns to the menu after every operation including errors.	Minor lapses, such as one prompt that is not robust or numbering that deviates from Section 6.9.	Crashes or infinite loops on bad input; names with spaces unsupported.
Code organization and comments 5 points	Separate modules for facilities, hash table, graph, and tree; main.cpp only coordinates; file purpose and author comments and function comments present; no arbitrary fixed limits.	Some logic in main.cpp or sparse comments.	Monolithic code, no comments, or fixed-size arrays.
Documentation 5 points	README (purpose, build and run commands, layout, data-directory argument) and design document (hash function, collision strategy with justification, collision-counting definition, capacity, graph representation with justification, main classes) are accurate and concise.	Documents are present but missing items or inconsistent with the code.	Missing or unusable.
Testing evidence and demonstration 5 points	The [M1] tests are recorded with input, expected, actual, and result, including a collision and a duplicate-key attempt; live demonstration of all M1 features works.	Some required tests are missing or lack outcomes; the demonstration partly works.	No test evidence or the demonstration fails.
Repository, ZIP, and teamwork	ZIP and GitHub match and contain the required files with no	Minor mismatch, uneven commits, or thin contribution	ZIP or repository missing or materially different; a

Criterion (points)	Full credit	Partial credit	Little or no credit
evidence 5 points	executables; .gitignore is appropriate; all three members have meaningful commits; CONTRIBUTIONS.txt is complete.	details.	member's work is absent from the history.
Total	**100 points**		

12.2 Final Submission Rubric

Criterion (points)	Full credit	Partial credit	Little or no credit
Milestone 1 features still correct and integrated 15 points	All M1 features are available through the Final menu numbering and work correctly on the sample and additional data; earlier defects were fixed.	Most M1 features work; a few defects remain or menu numbering deviates.	Several M1 features are broken or missing.
File loading robustness and load summaries 10 points	Instructor test files (duplicates, reversed edges, bad IDs, unknown references, Windows line endings, empty and missing files) are handled with correct warnings and summaries and no crashes.	Most cases handled; some warnings or summaries are wrong, or one test file causes incorrect results.	Crashes or widespread incorrect loading on the additional files.
Name and category search 8 points	Complete-name and complete-category searches ignore case and outer whitespace; all matches display in file order with a count; zero matches show a message and count zero; same-name facilities are both shown.	Works in common cases but fails on capitalization, whitespace, or zero-match handling.	Missing, overwrites records, or returns incorrect matches.

Criterion (points)	Full credit	Partial credit	Little or no credit
Shortest path (BFS) 12 points	BFS with parent tracking; path printed one building per line with the connection count; same start and destination returns 0; unknown buildings identified; no-path message; ties follow neighbor insertion order (Library to Gym via Student Union).	Correct on simple cases, but ties, same-building, or no-path cases are wrong.	Not BFS, incorrect paths, or crashes.
Postorder, level-order, and statistics 12 points	Postorder and level-order are correct with siblings in insertion order; level-order uses a queue; total, leaves, and height are correct for sample, root-only, and empty trees.	Traversals work but statistics are wrong in an edge case, or level-order does not use a queue.	Traversals missing or incorrect; statistics hard-coded.
Facility category report 7 points	Categories grouped ignoring capitalization with first accepted spelling; counts and one-decimal percentages computed from loaded data; total shown; no-facilities case reports none and total zero without dividing by zero.	Report works on the sample but mishandles capitalization or the empty case.	Missing, hard-coded, or divides by zero.
Graph connectivity report 6 points	Total buildings and connections; every building tied for highest degree; isolated buildings or None; correct component count; uses the shared graph representation.	One element wrong (for example, only one building shown in a tie, or an incorrect component count in a special case).	Missing, hard-coded, or mostly incorrect.
Menu and error handling 7 points	All 17 Final menu options use the required numbering; invalid input, text at numeric prompts, names with spaces, and repeated operations after errors are all handled; every error-table case behaves as specified.	Minor lapses in recovery or messages.	Crashes, infinite loops, or missing menu options.

Criterion (points)	Full credit	Partial credit	Little or no credit
Code quality and memory management 6 points	Modular code with required comments; no arbitrary limits; destructors release tree nodes and other heap memory; a clean memory-check run is recorded if a checker is available.	Mostly modular, but some leaks or sparse comments.	Monolithic code, widespread leaks, or fixed-size limits.
Documentation 5 points	README and design document are updated and accurate (build and run, module overview, hash function, collision strategy, graph representation, tree, BFS, and reports).	Updated but incomplete or inconsistent with the code.	Not updated or unusable.
Testing evidence 7 points	All Section 10 tests are recorded with input, expected, actual, and result; edge cases are covered; the program compiles and runs successfully on CELL.	Most tests recorded; some edge cases missing or results not stated.	Little or no test evidence, or the program does not compile and run on CELL.
Demonstration, repository, and teamwork 5 points	Demonstration shows all features; ZIP and GitHub match; commits from every member; CONTRIBUTIONS.txt with individual reflections; a documented code review.	Demonstration or repository has minor gaps, or contribution evidence is thin.	Demonstration fails, ZIP and repository differ materially, or contribution evidence is missing.
Total	**100 points**		

12.3 Caps and Adjustments (Both Submissions)

- **Does not compile and run on CELL:** the submission is capped at 40 percent of its points.
- **Hard-coded records, paths, totals, or results:** zero for the affected criteria, and the work may be referred under the syllabus integrity policy.
- **Library hash container or scan used for ID lookup:** no credit for the hash-table items in the facilities and hashing criterion.
- **Crash during the demonstration** on a required feature: that feature is limited to partial credit.

- **Individual adjustment:** individual scores may differ from the team score based on contribution evidence (TEAM-5).
- **Late submissions** follow the syllabus and Canvas policy.

13. Common Mistakes to Avoid

- Using std::unordered_map, std::map, or std::hash instead of the required custom ID hash table.
- Scanning all facilities for ID lookup.
- Hashing the ID before normalizing it to uppercase, so F004 and f004 land in different slots.
- Overwriting a facility when another facility has the same name.
- Counting an undirected connection twice, or sorting neighbors and changing BFS order.
- Marking BFS vertices visited only when dequeued.
- Implementing a binary tree instead of a general tree.
- Asking for a parent when the department tree is empty.
- Treating categories that differ only by capitalization as different.
- Using cin >> name when names can contain spaces, or mixing cin >> and getline without discarding the leftover newline.
- Failing to trim the carriage return \r from files with Windows line endings.
- Failing to recover from invalid numeric input.
- Hard-coding sample records, paths, totals, or outputs.
- Crashing on missing files, empty structures, or failed searches.
- Changing the menu numbering from Section 6.9.
- Leaving tree nodes allocated at exit (Final).
- Submitting a ZIP that differs from the GitHub repository, or committing compiled executables.

14. Submission Checklists

14.1 Milestone 1

- The project compiles and runs successfully on the UNT CSE CELL machines, tested well before the deadline.
- The four files are in data/, load in the required order, and print warnings and load summaries.
- Invalid records are rejected with useful warnings; missing files and empty structures do not crash the program.
- Facilities display in file order and ID search uses the custom hash table; a collision has been demonstrated with the team's own test data.
- The custom hash function and one collision strategy are implemented and documented.
- The graph is undirected, preserves neighbor order, and counts each connection once.
- BFS returns the correct order, distances, and reachability count.
- The department structure is a general tree with add, search, hierarchy display, and preorder.
- The menu uses the M1 numbering, and invalid input does not crash or trap the program.
- README, design document, test_results.txt, and CONTRIBUTIONS.txt are complete.

- ZIP archive and GitHub repository contain matching files; all members have commits.

14.2 Final Submission

- All Milestone 1 items above still hold, with the Final menu numbering.
- Name and category searches and both reports calculate results from loaded data.
- Shortest path handles same start and destination, unknown buildings, ties, and disconnected buildings.
- Postorder, level-order (with a queue), and department statistics work, including empty and root-only trees.
- Memory is released in destructors.
- All Section 10 tests are recorded in test_results.txt.
- README and design document are updated; CONTRIBUTIONS.txt includes reflections and a documented code review.
- ZIP archive and GitHub repository match and contain no executables.

Academic Integrity

All submitted work must be created entirely by your group.

You may NOT:

• Copy code from other students or groups
• Download project solutions
• Use AI-generated code (ChatGPT, Copilot, Gemini, Claude, etc.)
• Use code obtained from online sources without instructor permission

Discussion of concepts and algorithms is permitted.

Sharing source code, project files, or AI-generated implementations is prohibited and will be treated as academic dishonesty.

Failure to compile, inability to explain submitted code, or violations of the Academic Integrity policy may result in significant grade deductions and may be referred for further academic review.

All group members must be able to explain the code they submit.

Final Advice

Start early.

Most project failures occur because groups:

- delay integration
- fail to test incrementally
- wait until the last week to combine code

Build and test your system gradually throughout the project timeline.